Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация с использованием кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26(1)

Пасевич К.Ю.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE).

Ход работы

1. Модифицировать программу из ЛР №1:

- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.

2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:

- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.

4. Провести сравнительный анализ:

    - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
        - MSE + фиксированный шаг (данные из ЛР №1);
        - MSE + адаптивный шаг (данные из ЛР №2);
        - BCE + фиксированный шаг;
        - BCE + адаптивный шаг.
    - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).

5. Визуализировать результаты:

    - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
    - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
    - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.

6. Реализовать режим функционирования:

    - Пользователь вводит координаты x1,x2.

- Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
- Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).

7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Вариант 9

x_1	x_2	e
2	1	0
-2	1	1
2	-1	0
-2	-1	0

Код программы:

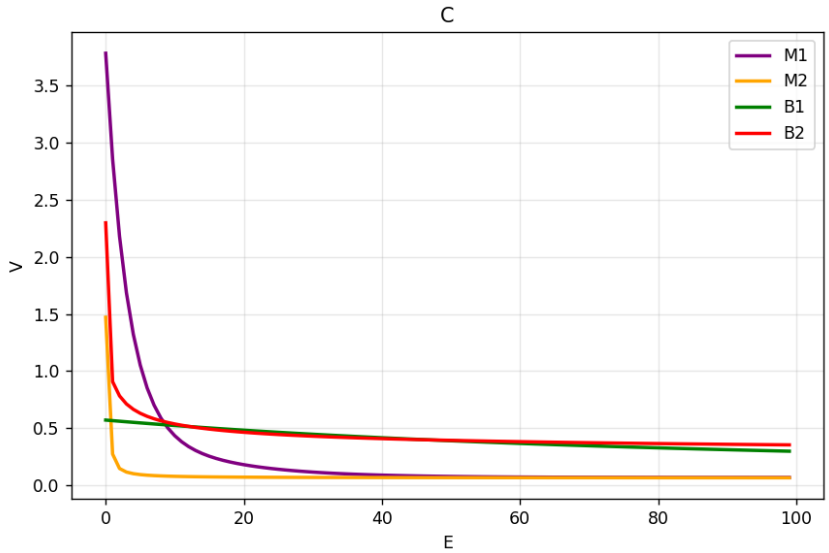
```
import numpy as np
import matplotlib.pyplot as plt

A = np.array([
    [2, 1],
    [-2, 1],
    [2, -1],
    [-2, -1]
], dtype=float)

B = np.array([0, 1, 0, 0], dtype=float)

C = 100
D = 0.01

def E(F):
    return 1 / (1 + np.exp(-F))

def G(H, I):
    J = 1e-9
    return -(I*np.log(H+J) + (1-I)*np.log(1-H+J))

def K(L, M, N):
    O = np.random.randn(2)
    P = np.random.randn()
    Q = []

    for R in range(C):
        S = 0
```

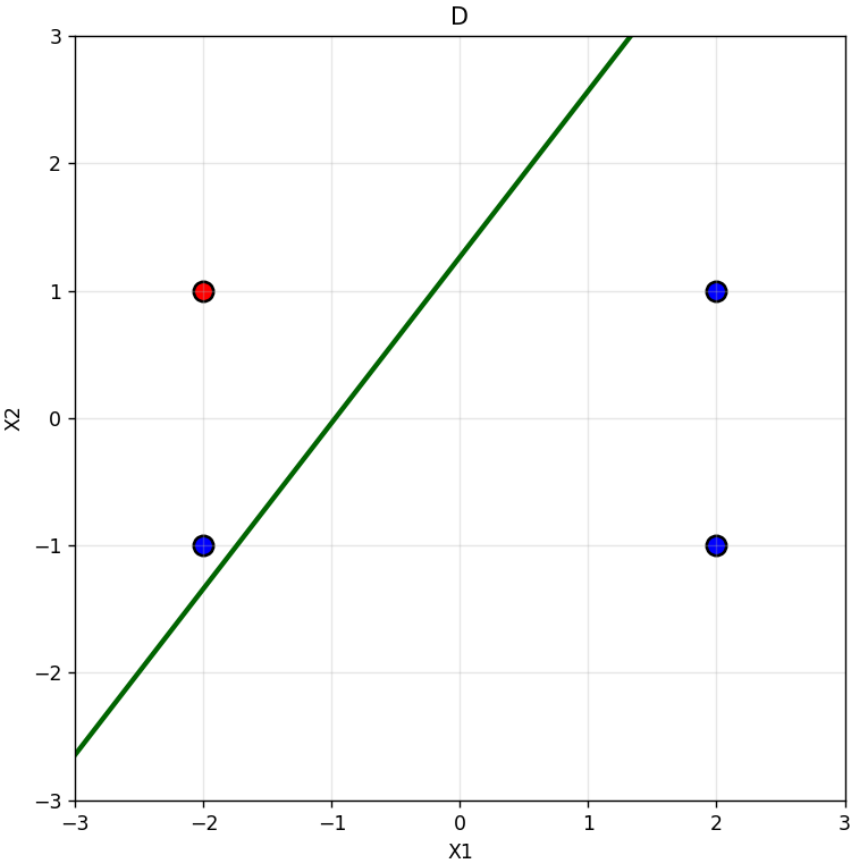
```
        for T, U in zip(L, M):
            V = np.dot(O, T) - P
            W = V
            X = U - W
            S += X**2

            O += N * X * T
            P -= N * X

        Q.append(S/len(L))
        if Q[-1] <= D:
            break

    return O, P, Q

def Y(Z, AA):
    O = np.random.randn(2)
    P = np.random.randn()
    Q = []
    AB = 1

    for R in range(C):
        AC = 0
        for T, U in zip(Z, AA):

            AD = 0.5 / AB
            AB += 1

            V = np.dot(O, T) - P
            W = V
            X = U - W
            AC += X**2

            O += AD * X * T
            P -= AD * X

        Q.append(AC/len(Z))
        if Q[-1] <= D:
            break

    return O, P, Q

def AE(AF, AG, AH):
    O = np.random.randn(2)
    P = np.random.randn()
    Q = []

    for R in range(C):
        AI = 0
        for T, U in zip(AF, AG):

            V = np.dot(O, T) - P
            W = E(V)

            AI += G(W, U)

            AJ = (W - U)
            O -= AH * AJ * T
            P += AH * AJ
```

```
        Q.append(AI/len(AF))
        if Q[-1] <= D:
            break

    return O, P, Q

def AK(AL, AM):
    O = np.random.randn(2)
    P = np.random.randn()
    Q = []
    AN = 1

    for R in range(C):
        AO = 0
        for T, U in zip(AL, AM):

            AP = 1 / AN
            AN += 1

            V = np.dot(O, T) - P
            W = E(V)

            AO += G(W, U)

            AQ = (W - U)
            O -= AP * AQ * T
            P += AP * AQ

        Q.append(AO/len(AL))
        if Q[-1] <= D:
            break

    return O, P, Q

AR, AS, AT = K(A, B, 0.01)
AU, AV, AW = Y(A, B)
AX, AY, AZ = AE(A, B, 0.01)
BA, BB, BC = AK(A, B)

plt.figure(figsize=(8,5))
plt.plot(AT, color='purple', linewidth=2, label="M1")
plt.plot(AW, color='orange', linewidth=2, label="M2")
plt.plot(AZ, color='green', linewidth=2, label="B1")
plt.plot(BC, color='red', linewidth=2, label="B2")
plt.xlabel("E")
plt.ylabel("V")
plt.title("C")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

plt.figure(figsize=(7,7))

for i in range(len(A)):
    BD = "red" if B[i] == 1 else "blue"
    plt.scatter(A[i,0], A[i,1], color=BD, s=100, edgecolors='black',
linewidth=1.5)
```

```
BE = np.linspace(-3, 3, 300)
if BA[1] != 0:
    BF = (BB - BA[0]*BE) / BA[1]
    plt.plot(BE, BF, color='darkgreen', linewidth=2.5)

plt.xlim(-3, 3)
plt.ylim(-3, 3)
plt.grid(True, alpha=0.3)
plt.title("D")
plt.xlabel("X1")
plt.ylabel("X2")
plt.show()

def BG(BH, BI):
    BJ = BA[0]*BH + BA[1]*BI - BB
    BK = E(BJ)
    BL = 1 if BK >= 0.5 else 0


    print(f"P1: {BK:.4f}")
    print(f"C: {BL}")


    plt.figure(figsize=(7,7))

    for i in range(len(A)):
        BM = "red" if B[i] == 1 else "blue"
        plt.scatter(A[i,0], A[i,1], color=BM, s=100, edgecolors='black',
linewidth=1.5)

    if BA[1] != 0:
        plt.plot(BE, BF, color='darkgreen', linewidth=2.5)
    plt.scatter(BH, BI, color="magenta", s=200, marker="X",
edgecolors='black', linewidth=2, zorder=5)

    plt.xlim(-3, 3)
    plt.ylim(-3, 3)
    plt.grid(True, alpha=0.3)
    plt.title("P")
    plt.xlabel("X1")
    plt.ylabel("X2")
    plt.show()

while True:
    BN = input("X1 (exit): ")
    if BN.lower() == "exit":
        break
    BO = input("X2: ")

    try:
        BG(float(BN), float(BO))
    except:
        print("E")
```

Результат программы:



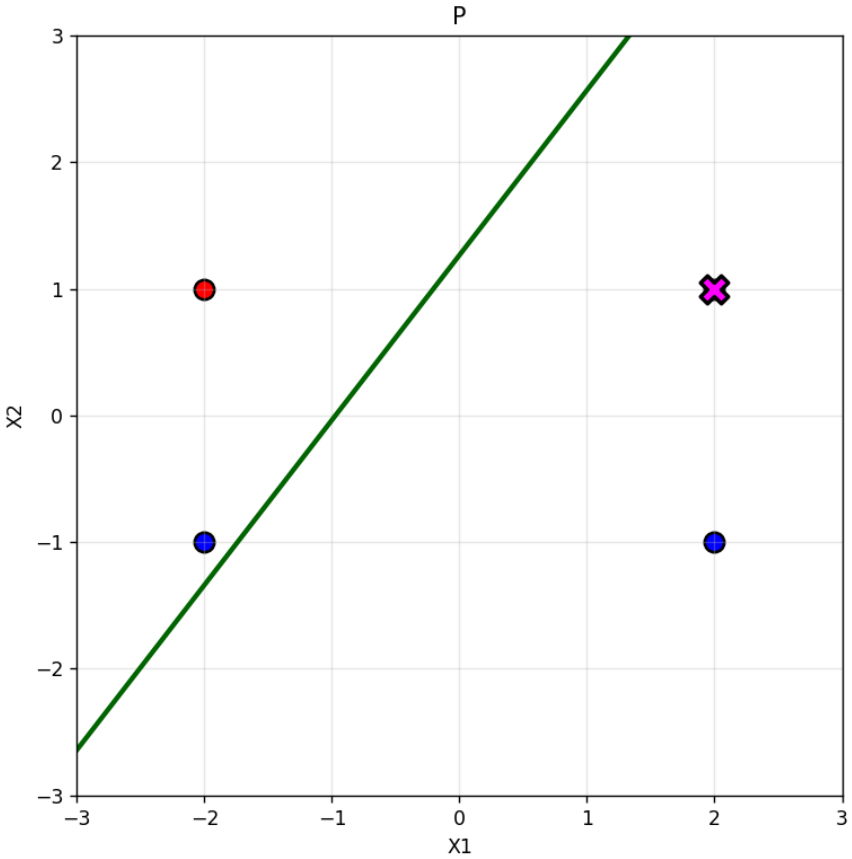


X1 (exit): 2

X2: 1

P1: 0.2109

C: 0



MSE с фиксированным шагом обучается быстрее BCE, потому что линейная активация позволяет весам меняться без ограничений, а сигмоида в BCE замедляет обучение, сжимая выход и уменьшая градиент. Адаптивный шаг в BCE эффективен: сначала резко снижает ошибку, потом плавно настраивает веса. Разделяющие линии отличаются: MSE минимизирует квадрат отклонений, BCE ищет вероятностное разделение классов. Для бинарной классификации BCE лучше, так как работает с вероятностями и сильнее штрафует уверенные ошибки, хотя учится медленнее. MSE проще и быстрее, но её выходы сложнее интерпретировать как вероятности.

Вывод: изучила принципы бинарной классификации с вероятностной точки зрения. Реализовала однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE).